Global Chronicle

VOL. 1, NO. 047

2026-03-30

FEATURES

Year in Review: 2020 into 2021

hello@taniarascia.com · Tania Rascia

Well, 2020, it's been a slice. I've been pretty AWOL lately on all things internet, and I can't decide if I have a lot to say or if I'd rather say nothing at all, but I've always made an end of the year post since I started this blog - going into 2017 , 2018 , 2019 , 2020 , so I'll keep up the tradition and write something today for 2021.

It's fun for me to look over those posts - I can see a zoomed out view of what I did and what I was focused on throughout each year. I wrote hundreds of articles, recorded the occasional song, made a lot of commits, built a few open-source projects, had a break up or three, made a bit of art, traveled all over Europe, and learned a ton.

This year is different though, right? A lot of things happened that we're all aware of, but one thing that happened was that life slowed down a bit. No more commuting to work for me, no more conferences, a lot more spare time. A lot more time on the internet for everyone.

I've spent so long focusing on being productive, feeling like I have to be productive all the time. I made it a point to never make promises or obligations about my creative output, but I've always tried to put out a consistent stream of quality material - at least a good tutorial once a month for the past five years or so. I felt good and productive if I produced something for the web and learned something new and wrote about it.

But that's five years of working 8 hours every day on code during my day job, and often spending the rest of my evening working on articles for DigitalOcean, for this website, for other publications, making my own open-source projects, talking about code on Twitter, and feeling like I should accept or consider all the speaking engagements and podcast requests and anything else that comes my way, because if I start turning them all down, the momentum I'm creating will disappear. (Fortunately I've usually kept "HELL YEAH!" or "no." by Derek Sivers in mind when responding to things.)

It started getting to the point where I was dreading code, dreading speaking engagements, and the last thing I wanted to do was scroll through the Twitter or Reddit feed and read little arguments about this or that framework intermixed with political outrage, or maintain dozens of open-source GitHub repositories for tutorials that are slowly getting out of date. Not to mention all the emails and comments that would come in, which could be anything from a nice email from a thoughtful person who would become a new friend, to people emailing me their coding questions as if I'm Google, people attempting to shut down or critique every aspect of an article, and of course plenty of "are you single?" and "you're such a good coder for a girl" type emails, or worse things I'd rather not mention.

Of course, the vast majority is positive, but it's always the negative stuff that lingers and bothers you throughout the day, and regardless of good or bad, it all requires mental energy. Even the good is odd to me, because often people will have an idea on me based what little I've put out into the world, and if you came to know me as the impatient, flawed and complex human being I am, it likely would not align and reality might possibly disappoint you. If anything, I'm more scared and hesitant of praise than critiques.

So, I've been burnt out on all code and community after several years of my life revolving around it completely. I'm far from famous, but with nearly 10,000 followers on GitHub and 15,000 on Twitter, I can't just say anything or put anything out there without lots of eyes on it, and it puts more pressure on me than if nobody was paying attention to anything I did.

Interestingly, when I opt out of all of this and only pay attention to the real world, it completely disappears. None of my friends or family really have any idea of anything I've done online. They know I do some coding stuff, but that's where it ends, and that's reality.

There are amazing people all around the world and the internet still gives us so many opportunities to connect and create and help others and do wonderful things, but there is so much I don't like about the way things are right now: being constantly bombarded by ads and distractions, infinite scroll, algorithmic manipulation, addiction, corporate interests, dopamine hits from increasing follows and shares and influence, impatience and a lack of focus, negativity, fear and paranoia from always hearing about all the worst things that are happening in the world, conspiracies and a lack of critical thinking. I just don't want to be part of this and affected by it as much as I can, I want to reject it all and go against the grain and live a slower, simpler life.

One small thing I can do is encourage a few other people to also go against the grain and read a book instead of scrolling a feed, and focus on a few deeper relationships as opposed to collecting and quantifying strangers. I'm learning to be my own best friend. I deleted all my tweets and stopped scrolling through Twitter, I removed my email from any easy-to-find spot, and I make sure not to log into anything like Reddit. Without being tailored to my interests, you'll find that most of these sites are pretty boring and it's easy to just check something out for a few minutes and move on.

I found the latest three articles on The Raptitude interesting and helpful, so I'll share them with you.

How to Handle the Beast - it was helpful for me to read this and know that other people struggle with a lack of productivity and drive, and that it's okay and normal.

News is the Last Thing We Need Right Now - real news, fake world.

Own the Tools - why it might be a better idea to - for example - open a physical dictionary to find a word as opposed to

Using OAuth with PKCE Authorization Flow (Proof Key for Code Exchange)

hello@taniarascia.com · Tania Rascia

If you've ever created a login page or auth system, you might be familiar with OAuth 2.0 , the industry standard protocol for authorization. It allows an app to access resources hosted on another app securely. Access is granted using different flows, or grants, at the level of a scope.

For example, if I make an application (Client) that allows a user (Resource Owner) to make notes and save them as a repo in their GitHub account (Resource Server), then my application will need to access their GitHub data. It's not secure for the user to directly supply their GitHub username and password to my application and grant full access to the entire account. Instead, using OAuth 2.0, they can go through an authorization flow that will grant limited access to some resources based on a scope, and I will never have access to any other data or their password.

Using OAuth, a flow will ultimately request a token from the Authorization Server , and that token can be used to make all future requests in the agreed upon scope.

Note : OAuth 2.0 is used for authorization , (authZ) which gives users permission to access a resource. OpenID Connect, or OIDC, is often used for authentication , (authN) which verifies the identity of the end user.

The type of application you have will determine the grant type that will apply.

Grant Type Application type Example

Client Credentials Machine A server accesses 3rd-party data via cron job

Authorization Code Server-side web app A Node or Python server handles the front and back end

Authorization Code with PKCE Single-page web app/mobile app A client-side only application that is decoupled from the back end

For machine-to-machine communication, like something that cron job on a server would perform, you would use the Client Credentials grant type, which uses a client id and client secret. This is acceptable because the client id and resource owner are the same, so only one is needed. This is performed using the /token endpoint.

For a server-side web app, like a Python Django app, Ruby on Rails app, PHP Laravel, or Node/Express serving React, the Authorization Code flow is used, which still uses a client id and client secret on the server side, but the user needs to authorize via the third-party first. This is performed using both an /authorize and /token endpoints.

However, for a client-side only web app or a mobile app, the Authorization Code flow is not acceptable because the client secret cannot be exposed, and there's no way to protect it. For this purpose, the Proof Key for Code Exchange (PKCE) version of the authorization code flow is used. In this version, the client creates a secret from scratch and supplies it after the authorization request to retrieve the token.

Since PKCE is a relatively new addition to OAuth, a lot of authentication servers do not support it yet, in which case either a less secure legacy flow like Implicit Grant is used, where the token would return in the callback of the request, but using Implicit Grant flow is discouraged. AWS Cognito is one popular authorization server that supports PKCE.

PKCE Flow

The flow for a PKCE authentication system involves a user , a client-side app , and an authorization server , and will look something like this:

The user arrives at the app 's entry page

The app generates a PKCE code challenge and redirects to the authorization server login page via /authorize

The user logs in to the authorization server and is redirected back to the app with the authorization code

The app requests the token from the authorization server using the code verifier/challenge via /token

The authorization server responds with the token, which can be used by the app to access resources on behalf of the user

So all we need to know is what our /authorize and /token endpoints should look like. I'll go through an example of setting up PKCE for a front end web app.

GET /authorize endpoint

The flow begins by making a GET request to the /authorize endpoint. We need to pass some parameters along in the URL, which includes generating a code challenge and code verifier .

Parameter Description

response_type code

client_id Your client ID

redirect_uri Your redirect URI

code_challenge Your code challenge

code_challenge_method S256

scope Your scope

state Your state (optional)

We'll be building the URL and redirecting the user to it, but first we need to make the verifier and challenge.

The first step is generating a code verifier, which the PKCE spec defines as:

Automatically Detecting Text Encodings in C++

Jeff Preshing

Consider the lowly text file.

This text file can take on a surprising number of different formats. The text could be encoded as ASCII , UTF-8 , UTF-16 (little or big-endian), Windows-1252 , Shift JIS , or any of dozens of other encodings. The file may or may not begin with a byte order mark (BOM) . Lines of text could be terminated with a linefeed character \n (typical on UNIX), a CRLF sequence \r\n (typical on Windows) or, if the file was created on an older system, some other character sequence .

Sometimes it's impossible to determine the encoding used by a particular text file. For example, suppose a file contains the following bytes:

A2 C2 A2 C2 A2 C2

This could be:

a UTF-8 file containing "¢¢¢"

a little-endian UTF-16 (or UCS-2) file containing "□□□"

a big-endian UTF-16 file containing "□□□"

a Windows-1252 file containing "Â¢Â¢Â¢"

That's obviously an artificial example, but the point is that text files are inherently ambiguous. This poses a challenge to software that loads text.

It's a problem that has been around for a while . Fortunately, the text file landscape has gotten simpler over time, with UTF-8 winning out over other character encodings. More than 95% of the Internet is now delivered using UTF-8. It's impressive how quickly that number has changed; it was less than 10% as recently as 2006 .

UTF-8 hasn't taken over the world just yet, though. The Windows Registry editor, for example, still saves text files as UTF-16. When writing a text file from Python, the default encoding is platform-dependent; on my Windows PC, it's Windows-1252. In other words, the ambiguity problem still exists today. And even if a text file is encoded in UTF-8, there are still variations in format, since the file may or may not start with a BOM and could use either UNIX-style or Windows-style line endings.

How the Plywood C++ Framework Loads Text

Plywood is a cross-platform open-source C++ framework I released two months ago . When opening a text file using Plywood, you have a couple of options:

If you know the exact format of the text file ahead of time, you can call FileSystem::openTextForRead() , passing the expected format in a TextFormat structure.

If you don't know the exact format, you can call FileSystem::openTextForReadAutodetect() , which will attempt to detect the format automatically and return it to you.

The input stream returned from these functions never starts with a BOM, is always encoded in UTF-8, and always terminates each line of input with a single carriage return \n , regardless of the input file's original format. Conversion is performed on the fly if needed. This allows Plywood applications to work with a single encoding internally.

Here's how Plywood's automatic text format detection currently works:

Does the file start with a BOM? Use BOM encoding Can the file be decoded as UTF-8 without any errors or control codes? Use UTF-8 When decoding as UTF-8, are there decoding errors in more than 25% of non-ASCII code points? The 8-bit format is UTF-8 The 8-bit format is plain bytes Try decoding as little and big-endian UTF-16, then take the best score between those and the 8-bit format Detect line ending type (LF or CRLF) Done

yes yes yes no no no

Plywood analyzes up to the first 4KB of the input file in order to guess its format. The first two checks handle the vast majority of text files I've encountered. There are lots of invalid byte sequences in UTF-8, so if a text file can be decoded as UTF-8 and doesn't contain any control codes, then it's almost certainly a UTF-8 file. (A control code is considered to be any code point less than 32 except for tab, linefeed and carriage return.)

It's only when we enter the bottom half of the flowchart that some guesswork begins to happen. First, Plywood decides whether it's better to interpret the file as UTF-8 or as plain bytes. This is meant to catch, for example, text encoded in Windows-1252 that uses accented characters. In Windows-1252, the French word détail is encoded as 64 E9 74 61 69 6C , which triggers a UTF-8 decoding error since UTF-8 expects E9 to be followed be a byte in the range 80 - BF . After a certain number of such errors, Plywood will favor plain bytes over UTF-8.

After that, Plywood attempts to decode the same data using the 8-bit format, little-endian UTF-16 and big-endian UTF-16. It calculates a score for each encoding as follows:

Each whitespace character decoded is worth +2.5 points. Whitespace is very helpful to identify encodings, since UTF-8 whitespace can't be recognized in UTF-16, and UTF-16 whitespace contains control codes when interpreted in an 8-bit encoding.

ASCII characters are worth +1 point each, except for control codes.

Decoding errors incur a penalty of −100 points.

Control codes incur a penalty of −50 points.

Building an JavaScript Keyboard Accordion (the Musical Kind)

hello@taniarascia.com · Tania Rascia

It's been a while since I've written anything due to some personal concerns that I might write about later, but don't worry, I'm still around and I'm still coding. Recently, I went to Texas and bought a three-row diatonic button accordion. Diatonic accordions are popular for a lot of different types of folk music, which is generally learned by ear. This is good for me, because I don't really know how to read music anyway.

The accordion has 34 buttons on the treble side and 12 buttons on the bass side. Unlike a piano accordion, which has the same logical, chromatic layout as a piano, the diatonic accordion just has a bunch of buttons and I didn't really know where to start. Also, every note is different whether you're pulling the bellows out or pushing them in, so there are actually 68 notes on the treble side (albeit some are repeated). Also, as I'm sure you might be aware, accordions are loud. Very loud. In order to not piss off my neighbors too much, and to learn how the layout of this box works, I decided to make a little web app.

That web app is KeyboardAccordion.com , which like everything else I create in my free time is open source . I noticed that there are just enough keys on a computer keyboard to correspond to the accordion layout, and they're arranged in a similar pattern. With this, I can keep track of the notes, scales, and chords and start figuring out how to put it all together.

Here's what one of the accordions looks like:

I decided to make this app in Svelte, because I've used React and Vue professionally but have no experience with Svelte whatsoever and wanted to know what everyone loves about it.

Web Audio API

KeyboardAccordion.com only has one dependency, and that's Svelte. Everything else is done using plain JavaScript and the built-in browser Web Audio API . I'd never really used the Web Audio API before, so I figured out what I needed to to get this working.

The first thing I did was create an AudioContext and attach a GainNode , which controls the volume.

const audio = new (window . AudioContext || window . webkitAudioContext) ( ) const gainNode = audio . createGain ( ) gainNode . gain . value = 0.1 gainNode . connect (audio . destination)

As I was figuring everything out, I was experimenting with making new AudioContext for every note because I was trying to fade out the sound, but then I kept realizing that after 50 notes, the app would stop working. Fifty is apparently the limit for the browser, so it's better to just make one AudioContext for the entire app.

I'm using waves with the Audio API and not using any sort of audio sample, and I used the OscillatorNode to make each note. There are various types of waves you can use - square , triangle , sine , or sawtooth , which all have a different type of sound. I went with the sawtooth for this app because it worked out the best. Square makes an extremely loud, chiptune-esque sound like an NES which is kind of nice in its own way. Sine and triangle were a bit more subdued but if you don't fade the sound out properly, it makes a really unpleasant kind of cutting sound due to how your ear reacts when a wave gets cut off.

So for each note, I'd make an oscillator, set the wave type, set the frequency, and start it. Here's an example using 440 , which is a standard tuning for "A".

const oscillator = audio . createOscillator ( ) oscillator . type = 'sawtooth' oscillator . connect (gainNode) oscillator . frequency . value = 440 oscillator . start ( )

If you do that, the note will just play until infinity, so you have to make sure you stop the oscillator when you want the note to end.

oscillator . stop ( )

For me, this meant event listeners on the DOM that would listen for a keypress event to see if any button was pressed, and a keyup event to determine when any button was no longer being pressed. In Svelte, that's handled by putting event listeners on svelte:body .

So that's really everything there is to the Web Audio API itself when it comes to setting up the app - creating an AudioContext , adding a Gain , and starting/stopping an Oscillator for each note.

You could paste this into the console and it'll play a note. You'll have to either refresh or type oscillator.stop() to make it stop.

const audio = new (window . AudioContext || window . webkitAudioContext) ( ) const gainNode = audio . createGain ( ) gainNode . gain . value = 0.1 gainNode . connect (audio . destination)

const oscillator = audio . createOscillator ( ) oscillator . type = 'sawtooth' oscillator . connect (gainNode) oscillator . frequency . value = 440 oscillator . start ( )

I had to figure out how I wanted to lay out the data structure for this application. First of all, if I'm going to be using the Web Audio API with frequencies directly, I had to collect all of them.

Here's a nice map of notes to frequencies with all 12 notes and 8-9 octaves for each note, so I can use A[4] to get the 440 frequency.

tone

export const tone = {

From Hadoop and Cassandra to Kafka Streams

Nicolas Yann Couturier · Finn.no Tech

Some context

People who publish their classified ads on FINN.no get access to various statistics to see how their ads are performing. For a user it can look something like this:

Statistics the owner of a realestate ad gets to see

The top left bar chart shows the repartition of the incoming traffic by day and the legend to the right of it shows the total numbers for each type of incoming traffic. The lower section is divided in 3 parts:

the left part shows the number of views by unique users;

the one to the center shows how many users have been notified of the ad by email and how many added the ad to their favorites;

the right part shows how many viewers out of the total come from a specific type of traffic.

This gives the user basic insight into the reach of their ad, such as how many views it has and how many unique users have viewed it.

Around November 2016 there was a request to show more detailed information about the viewers, such as the age, gender and location distribution of the viewers (demographic information) so the owner could potentially change an ad to better fit the audience they wanted.

Existing solution

As users view ads, do actions (such as send a message to an ad's owner or scroll down and read the whole page, for example), these actions are gathered and published internally on Apache Kafka . These streams of data then become the basis for computing the statistics above. The plan was then to also publish demographic data about the viewers (such as location, age and gender) on Kafka and join the user actions with this demographic data to provide enhanced statistics.

At the time, the action events published on Kafka were saved to Cassandra Apache Cassandra clusters, and statistics were being computed as batches on an aging Apache Hadoop cluster reading from Cassandra. Both our Hadoop and Cassandra clusters had not receive much love recently and were all on end-of-life versions. The old system also had an increasing tendency to fail, so we were also in the process to decide to either spend our time to refresh/renew the old system or replace it.

A simplified overview of the previous architecture

Replacement solution

We first tried to find a good way of joinining the latest demographic data being received through Kafka as part of the existing statistics computation done with Hadoop. We quickly learned that efficient communication with Kafka from a Hadoop job was not easy to get right. After some experimentation we had a solution that sort of worked, but it was complicated and far from elegant.

After doing some research we started to look at the possibility of using Kafka Streams . Reading the introductory blog post - especially the "Simple is Beautiful" paragraph - it felt like Streams was just what we needed.

The most interesting aspects were:

no more external runtime for jobs (in our case YARN/ Hadoop);

no more intermediary external storage (Cassandra);

built-in fault tolerance;

only POJOs running as normal Java apps;

limiting the number of components by using Kafka for the whole solution.

This would lead to less moving parts, less maintenance and what we saw as a simpler and more elegant solution.

We also evaluated other solutions, amongst which:

upgrade our Hadoop and Cassandra clusters;

Apache Spark micro-batches on top of an Apache Parquet storage filled from Kafka;

hybrid solutions with hyperloglog and other similar cardinality tricks to estimate the unique users visiting an ad.

We were already using Kafka (and it is seen as a central part of our future architecture), so Streams was deemed to be the most attractive and straight-forward solution.

Streams in use

The first prototype took less than a week to get up and running, and from there it took 6 months to develop a fully operational, properly sized version running in production on our Kubernetes container cluster.

As part of development we also wrote tools for monitoring, error handling/diagnostics, reprocessing and for migration of the existing statistics out of the old Cassandra storage and into our new storage. During this development, the original plan to join in demographics data was put on hold (for non-technical reasons) and the project turned into a more technical one: move the statistics system out of the aging Hadoop and Cassandra clusters and migrate to something that incurs less maintenance and fits better with our overall technology strategy.

QA role is dead but development and testing together provides quality for FINN.no

vivek · Finn.no Tech

When a developer is finished with a development issue, it must be tested. People often think the goal of testing should be to find all the errors. Well, it's not entirely realistic. Actually, it's impossible! Despite the title of "quality insurance" testers can never really ensure level of quality, nor can they be expected to find all the errors. The real goal of testing should be to improve the software. As at the end of the day we want to minimize the cost of fixing critical bugs after a release, minimize cost of handling customer complaints and pushing a patch in the production. So it is import to understand that Quality Assurance is a process, not a department.

Today there are basically two different components of QA role in FINN. One of which is "QA in a team" and the other one is of a "team without the QA". The QA role today is often a safety net for many developers and teams. The role involves manual testing in the sprint, regression testing and writing Cucumber tests. In addition this role has some administrative tasks such as being contact person for the Release Manager, participating in the release status meeting, filling out the checklist of early / late shift, contact for Tech Support (E-journal issues), participating in status meetings at Call Center after release, and verifying the patch to production. Everything that's listed so far is managed also by teams without QA. What we have seen so far, is that a team without QA delivers the same quality as the team with QA in developing an FINN application.

Then comes the question, how do they manage to do it?

They have the same responsibilities as the "QA in a team", but they have different practices on how they manage the tasks (some have rolling QA role, others have a fixed arrangement but they are NOT dependent on one individual person).

If we want to deliver a good quality code, testing must become a central pillar of the development process. When I say the role of QA is dead, but development and testing together provides quality for FINN, by that I mean that the whole team should focus on unit testing, integration testing, system testing and functional testing (as part of the pipeline). This will build and provide quality from top to bottom in the product stack.

Despite back-end test automation, front end problems with web applications are always discovered by human testers, including issues related to browser compatibility or CSS flaws. This can be covered by focusing on exploratory testing, which is also done in a team with no QA.

The QA testers should rather concentrate on the planning of tests within their teams. They should focus on three things.

Attributes: Describes the high level concept testing are meant to ensure performance testing, usable testing, secure testing, accessible testing and so forth.

Components: Define the major code chunks that comprise the product. These are classes, module names and features of the application.

Capabilities: Describes user actions and activities.

We know it very well that we cannot test everything so there is no pointing in investing too much time in documenting everything either? And we know as we start testing things the schedule, the requirements, the architecture etc are going to change. But the greater benefit is that whole team takes the responsibility for the quality, quicker feedback, tests documenting the code and not the least satisfied customers.

The QA role today is often a safety net for many developers and teams.

vivek

Litestream Writable VFS

Fly.io Blog

I'm Ben Johnson, and I work on Litestream at Fly.io. Litestream is the missing backup/restore system for SQLite. It's free, open-source software that should run anywhere, and you can read more about it here .

Each time we write about it, we get a little bit better at golfing down a description of what Litestream is. Here goes: Litestream is a Unix-y tool for keeping a SQLite database synchronized with S3-style object storage. It's a way of getting the speed and simplicity wins of SQLite without exposing yourself to catastrophic data loss. Your app doesn't necessarily even need to know it's there; you can just run it as a tool in the background.

It's been a busy couple weeks!

We recently unveiled Sprites . If you don't know what Sprites are, you should just go check them out . They're one of the coolest things we've ever shipped. I won't waste any more time selling them to you. Just, Sprites are a big deal, and so it's a big deal to me that Litestream is a load-bearing component for them.

Sprites rely directly on Litestream in two big ways.

First, Litestream SQLite is the core of our global Sprites orchestrator. Unlike our flagship Fly Machines product, which relies on a centralized Postgres cluster, our Elixir Sprites orchestrator runs directly off S3-compatible object storage. Every organization enrolled in Sprites gets their own SQLite database, synchronized by Litestream.

This is a fun design. It takes advantage of the "many SQLite databases" pattern, which is under-appreciated. It's got nice scaling characteristics. Keeping that Postgres cluster happy as Fly.io grew has been a major engineering challenge.

But as far as Litestream is concerned, the orchestrator is boring, and so that's all I've got to say about it. The second way Sprites use Litestream is much more interesting.

Litestream is built directly into the disk storage stack that runs on every Sprite.

Sprites launch in under a second, and every one of them boots up with 100GB of durable storage. That's a tricky bit of engineering. We're able to do this because the root of storage for Sprites is S3-compatible object storage, and we're able to make it fast by keeping a database of in-use storage blocks that takes advantage of attached NVMe as a read-through cache. The system that does this is JuiceFS, and the database — let's call it "the block map" — is a rewritten metadata store, based (you guessed it) on BoltDB.

I kid! It's Litestream SQLite, of course.

Everything in a Sprite is designed to come up fast.

If the Fly Machine underneath a Sprite bounces, we might need to reconstitute the block map from object storage. Block maps aren't huge, but they're not tiny; maybe low tens of megabytes worst case.

The thing is, this is happening while the Sprite boots back up. To put that in perspective, that's something that can happen in response to an incoming web request; that is, we have to finish fast enough to generate a timely response to that request. The time budget is small.

To make this even faster, we are integrating Litestream VFS to improve start times. The VFS is a dynamic library you load into your app. Once you do, you can do stuff like this:

Wrap text

Copy to clipboard

sqlite> .open file:///my.db?vfs = litestream sqlite> PRAGMA litestream_time = '5 minutes ago' ; sqlite> SELECT * FROM sandwich_ratings ORDER BY RANDOM () LIMIT 3 ; 22| Veggie Delight|New York|4 30|Meatball|Los Angeles|5 168| Chicken Shawarma Wrap|Detroit|5

> *Most storage block maps are much smaller than this, but still.*
>
> Fly.io Blog

Litestream VFS lets us run point-in-time SQLite queries hot off object storage blobs, answering queries before we've downloaded the database.

This is good, but it's not perfect. We had two problems:

We could only read, not write. People write to Sprite disks. The storage stack needs to write, right away.

Running a query off object storage is a godsend in a cold start where we have no other alternative besides downloading the whole database, but it's not fast enough for steady state.

These are fun problems. Here's our first cut at solving them.

Writable VFS

The first thing we've done is made the VFS optionally read-write. This feature is pretty subtle; it's interesting, but it's not as general-purpose as it might look. Let me explain how it works, and then explain why it works this way.

Keep in mind as you read this that this is about the VFS in particular. Obviously, normal SQLite databases using Litestream the normal way are writeable.

The VFS works by keeping an index of (file,offset, size) for every page of the database in object storage; the data comprising the index is stored, in LTX files , so that it's efficient for us to reconstitute it quickly when the VFS starts, and lookups are heavily cached. When we queried

Log4j2 in production – making it fly

mick · Finn.no Tech

Now that log4j2 is the predominant logging framework in use at FINN.no why not share the good news with the world and try to provide a summary over the introduction of this exciting new technology into our platform.

let's just one logging framework

In beginning of September 2013 it became the responsibility of one of our engineering teams to introduce a "best practice for logging" for all of FINN.

The proposal put forth was that we can standardise on one backend logging framework while there would be no need to standardise on the abstraction layer directly used in our code.

The rationale to not standardising the logging abstractions was...

• nearly every codebase, through a tree of dependencies, already includes all the different logging abstraction libraries, so the hard work of configuring them against the chosen logging framework is still required, • the different APIs to the different logging abstractions are not difficult for programmers to go between from one project to another.

While the rationale to standardising the logging framework was...

• makes life easier for programmers with one documented "best practice" for all,

• makes it possible through an in-house library to configure all abstraction layers, creating less configuration for programmers, • makes life easier for operations knowing all jvms log the same way,

• makes life easier for operations knowing all log files follow the same format.

Log4j2 wins HANDS down

Log4j2 was chosen as the logging framework given... • it provided all the features that logback was becoming popular for, • between old log4j and logback it was the only framework that was written in java with modern concurrency (ie not hard synchronised methods/blocks), • it provided a significant performance improvement (1000-10000 times faster)

• it consisted of a more active community (logback has been announced as the replacement for the old log4j, but log4j2 saw a new momentum in its apache community).

This proposal was checked with a vote by finn programmers • 73% agreed, 27% were unsure, no-one disagreed.

when nightly compression jams

Earlier on in this process we hit a bug with the old log4j where nightly compression of already rotated logfiles were locking up all requests in any (or most) jvms for up to ten seconds. This fault came back to poor java concurrency code in the original log4j (which logback cloned). Exacerbated by us having scores of jvms for all our different microservices running on the same machines so that when nightly compression kicked in it did so all in parallel. Possible fixes here were to

a) stop compression of log files,

b) make loggers async, or

c) migrate over quickly to log4j2.

After some investigation, (c) was ruled out because there was no logstash plugin for log4j2 ready and moving forward without the json logfiles and the logstash & kibana integration was not an option. (a) was chosen as a temporary solution.

ready, steady, go...

Later on, when we started the work on upgrading all our services from thrift-0.6.1 up to thrift-0.9.1, we took the opportunity to kill two birds with the one stone. Log4j2 was out of beta, and we had ironed out the issues around the logstash plugin.

We'd be lying if we told you it was all completely pain free, introducing Log4j2 came with some concerns and hurdles.

• Using a release candidate of log4j2 in production lead to some concerns. So far the only consequence was slow startup times (eg even small services paused for 8 seconds during startup). This was due to log4j2 having to scan all classes for possible log4j plugins. This problem was fixed in 2.0-rc2. On the bright side – our use of the release candidate meant we spotted early and getting the upcoming initial release of log4j2 to support shaded jarfiles, of which we are heavily dependent on.

• Operation's had expressed concerns over nightly compression, raised from the earlier problem around nightly compression, that even if code no longer blocked while the compressing was happening in a background thread the amount of parallel compressions spawned would lead to IO contention which in turn leads to CPU contention. Because of this very real concern extensive tests have been executed, so far they've shown no measurable (under 1ms) impact exists upon services within the FINN platform. Furthermore this problem can be easily circumvented by adding the SizeBasedTriggeringPolicy to your appender, thereby enforcing a limit on how much parallel compression can happen at midnight.

• The new logstash plugin (which finn has actively contributed to on github) caused a few breakages to the format expected by our custom logstash parsers written by operations. Unfortunately this parser is based off the old log4j

Create impeccable MIME email from markdown

Joe Nelson (begriffs)

The goal

I want to create emails that look their best in all mail clients, whether graphical or text based. Ideally I'd write a message in a simple format like Markdown, and generate the final email from the input file. Additionally, I'd like to be able to include fenced code snippets in the message, and make them available as attachments.

I created a utility called mimedown that reads markdown through stdin and prints multipart MIME to stdout.

Let's see it in action. Here's an example message:

This is a demo email with code

Hey, does this code look fishy to you?

```crash.c #include

int main(void) { char a[] = "string literal"; char *p = "string literal";

/* capitalize first letter */ p[0] = a[0] = 'S'; printf("a: %s\np: %s\n", a, p); return 0; } ```

It blows up when I compile it and run it:

```compile.txt $ cc -std=c99 -pedantic -Wall -Wextra crash.c -o crash $ ./crash Bus error: 10 ```

Turns out we're invoking undefined behavior.

* The C99 spec, appendix J.2 Undefined Behavior mentions this case: > The program attempts to modify a string literal (6.4.5). * Steve Summit's C FAQ [question 1.32](http://c-faq.com/decl/strlitinit.html) covers the difference between an array initialized with string literal vs a pointer to a string literal constant. * The SEI CERT C Coding standard [STR30-C](https://wiki.sei.cmu.edu/confluence/display/c/STR30-C.+Do+not+attempt+to+modify+string+literals) demonstrates the problem with non-compliant code, and compares with compliant fixes.

After running it through the generator and emailing it to myself, here's how the result looks in the Fastmail web interface:

rendered in fastmail

Notice how the code blocks are displayed inline and are available as attachments with the correct MIME type.

I intentionally haven't configured Mutt to render HTML, so it falls back to the text alternative in the message, which also looks good. Notice how the message body is interleaved with Content-Disposition: inline attachments for each code snippet.

code and text in Mutt

The email generator also creates references for external urls. It substitutes the urls in the original body text with references, and consolidates the links into a bibliography of type text/uri-list at the end of the message. Here's another Mutt screenshot of the end of the message, with red circles added.

links as references

The generated MIME structure of our sample message looks like this:

I 1 [multipa/alternativ, 7bit, 3.1K] I 2 ├─> [multipa/mixed, 7bit, 1.7K] I 3 │ ├─> [text/plain, 7bit, utf-8, 0.1K] I 4 │ ├─>crash.c [text/x-c, 7bit, utf-8, 0.2K] I 5 │ ├─> [text/plain, 7bit, utf-8, 0.1K] I 6 │ ├─>compile.txt [text/plain, 7bit, utf-8, 0.1K] I 7 │ ├─> [text/plain, 7bit, utf-8, 0.5K] I 8 │ └─>references.uri [text/uri-list, 7bit, utf-8, 0.2K] I 9 └─> [text/html, 7bit, utf-8, 1.3K]

At the outermost level, the message is split into two alternatives: HTML and multipart/mixed. Within the multipart/mixed part is a succession of message text and code snippets, all with inline disposition. The final mixed item is the list of referenced urls (if necessary).

Other niceties

Lines of the message body are re-flowed to at most 72 characters, to conform to historical length constraints. Additionally, to accommodate narrow terminal windows, mimedown uses a technique called format=flowed . This is a clever standard ( RFC 3676 ) which adds trailing spaces to any lines that we would like the client reader to re-flow, such as those in paragraphs.

Neither hard wrapping nor format=flowed is applied to code block fences in the original markdown. Code snippets are turned into verbatim attachments and won't be mangled.

Finally, the HTML version of the message is tasteful and conservative. It should display properly on any HTML client, since it validates with ISO HTML (ISO/IEC 15445:2000, based on HTML 4.01 Strict).

Try it yourself

Clone it here: github.com/begriffs/mimedown . It's written in portable C99. The only build dependency is the cmark library for parsing markdown.

# Clarifications about Redis and Memcached

*Antirez*

If you know me, you know I'm not the kind of guy that considers competing products a bad thing. I actually love the users to have choices, so I rarely do anything like comparing Redis with other technologies.

However it is also true that in order to pick the right solution users must be correctly informed.

This post was triggered by reading a blog post published by Mike Perham, that you may know as the author of a popular library called Sidekiq, that happens to use Redis as backend. So I would not consider Mike a person which is "against" Redis at all. Yet in his blog post that you can find at the URL http://www.mikeperham.com/2015/09/24/storing-data-with-redis/ he states that, for caching, "you should probably use Memcached instead [of Redis]". So Mike simply really believes Redis is not good for caching, and he arguments his thesis in this way:

1) Memcached is designed for caching.

2) It performs no disk I/O at all.

3) It is multi threaded and can handle 100,000s of requests by scaling multi core.

I'll address the above statements, and later will provide further informations which are not captured by the above sentences and which are in my opinion more relevant to most caching users and use cases.

Memcached is designed for caching: I'll skip this since it is not an argument. I can say "Redis is designed for caching". So in this regard they are exactly the same, let's move to the next thing.

It performs no disk I/O at all: In Redis you can just disable disk I/O at all if you want, providing you with a purely in-memory experience. Except, if you really need it, you can persist the database only when you are going to reboot, for example with "SHUTDOWN SAVE". The bottom line here is that Redis persistence is an added value even when you don't use it at all.

It is multi threaded: This is true, and in my goals there is to make Redis I/O threaded (like in memcached, where the data access itself is not threaded, basically). However Redis, especially using pipelining, can serve an impressive amount of requests per second per thread (half a million is a common figure with very intensive pipelining. Without pipelining it is around 100,000 ops/sec). In the vanilla caching scenario where each Redis instance is the same, works as a master, disk ops are disabled, and sharding is up to the client like in the "memcached sharding model", to spin multiple Redis processes per system is not terrible. Once you do this what you get is a shared-nothing multi threaded setup so what counts is the amount of operations you can serve per single thread. Last time I checked Redis was at least as fast as memcached per each thread. Implementations change over time so the edge today may be of the one or the other, but I bet they provide near performances since they both tend to maximize the resources they can use. Memcached multi threading is still an advantage since it makes things simpler to use and administer, but I think it is not a crucial part.

There is more. Mike talks of operations per second without citing the *quality* of operations. The thing is in systems like Redis and Memcached the cost of command dispatching and I/O is dominating compared to actually touching the in-memory data structures. So basically in Redis executing a simple GET, a SET, or a complex operation like a ZRANK operation is about the same cost. But what you can achieve with a complex operation is a lot more work from the point of view of the application level. Maybe instead of fetching five cached values you can just send a small Lua script. So the actual "scalability" of the two systems have many dimensions, and what you can achieve is one of those.

Of Mike's concerns the only valid I can see is multi threading which, if we consider Redis in its special case of memcached replacement, may be addressed executing multiple processes, or simply by executing just one since it will be very very hard to saturate one thread doing memcached alike operations.

The real differences

—

Now it's time to talk about the *real* differences between the two systems.

* Memory efficiency

This is where Memcached used to be better than Redis. In a system designed to represent a plain string to string dictionary, it is simpler to make better use of memory. This difference is not dramatic and it's like 5 years I don't check it, but it used to be noticeable.

However if we consider memory efficiency of a long running process, things are a bit different. Read the next section.

But again to really evaluate memory efficiency, you should put into the bag that specially encoded small aggregated values in Redis are very memory efficient. For example sets of small integers are represented internally as an array of 8, 16, 32 or 64 bits integers, and are accessed in logarithmic time when you want to check the existence of some since they are ordered, so binary search can be used.

The same happens when you use hashes to represent objects instead of resorting to JSON. So the real memory efficiency must be evaluated with an use case at hand.

* Redis LRU vs Slab allocator

# Programming and Writing

*Antirez*

One year ago I paused my programming life and started writing a novel, with the illusion that my new activity was deeply different than the previous one. A river of words later, written but more often rewritten, I'm pretty sure of the contrary: programming big systems and writing novels have many common traits and similar processes.

The most obvious parallel between the two activities is that in both of them you write something. Code is not prose written in a natural language, yet it has a set of fixed rules (a grammar), certain forms that most programmers will understand as natural and others that, while formally correct, will sound hard to grasp.

There is, however, a much deeper connection between the two activities: a good program and a good novel are both the sum of local and global elements that work well. Good code must be composed of well written and readable single statements, but overall the different parts of the program must be orthogonal, designed in a coherent way, and have clean interactions. A good novel must also succeed in the same two scales of the micro and the macro. Sentences must be well written, but the overall structure and relationship between the parts is also crucial.

A less structural link between programming and writing is in the drive you need when approaching one or the other: to succeed you need to make progresses, and to make progresses you have to be consistent. There is extensive agreement on the fact that programs and novels don't write themselves, yet. Twenty years of writing code helped me immensely with this aspect; I knew that things happen only if you sit every day and write: one day one hundred words, the other day two thousands, but rare is the day I don't put words on the page. And if you have written code that is not just a "filler" for a bigger system, but a creation of your own, you know that writer block also happens in programming. The only difference is that for most people you are an engineer, hence, if you don't work, you are lazy. The same laziness, in the case of an artist, will assume the shape of a fascinating part of the creative process.

The differences.

I believe the most sharp difference between writing and programming is that, once written, edited and finalized, a novel remains immutable, mostly. There are several cases of writers returning on their novels after several years, publishing a bug fixed version of it, but this is rare and, even when happens, a one-shot process. Code evolves over time, is targeted by an endless stream of changes, often performed by multiple people. This simple fact has profound effects on the two processes: programmers often believe that the first version of a system can be quite imperfect; after all there will be time to make improvements. On the other hand writers know they have a single bullet for every novel, to the point that writing prose is mostly the act of rewriting. Rewriting sentences, whole chapters, dialogues that sound fake, sometimes two, three, or even ten times.

I believe programming, in this regard, can learn something from writing: when writing the first core of a new system, when the original creator is still alone, isolated, able to do anything, she should pretend that this first core is her only bullet. During the genesis of the system she should rewrite this primitive kernel again and again, in order to find the best possible design. My hypothesis is that this initial design will greatly inform what will happen later: growing organically something that has a good initial structure will result in a better system, even after years of distance from the original creation, and even if the original core was just a tiny faction of the future mass the system would eventually assume.

In case you are interested, a quick update about my sci-fi novel. After many self-reviews I sent the manuscript to my editor, Giulio Mozzi. He will send me the change proposals in a few weeks. I'll start a new review process informed by his notes, and hopefully finalize the novel in one or two months. Then, finally, I'll be ready to publish the Italian version. A the same time the finalized novel will be sent to my translator, in the US, and when she ends the translation the English version will be published as well. It's a long journey, but one that I deeply enjoyed taking. Comments

## Using Hapi.js, Mongoose, And MongoDB To Build A REST API

*Nic Raboy · Nic Raboy (polyglot developer)*

To continue on my trend of MongoDB with Node.js material, I thought it would be a good idea to use one of my favorite Node.js frameworks. Previously I had written about using Express.js with Mongoose , but this time I wanted to evaluate the same tasks using Hapi.js.

In this tutorial we're going to develop a simple RESTful API using Hapi.js , Joi and Mongoose as the backend framework, and MongoDB as the NoSQL database. Rather than just using Hapi.js as a drop in framework replacement, I wanted to improve upon what we had previously seen, by simplifying functions and validating client provided data.

The post Using Hapi.js, Mongoose, And MongoDB To Build A REST API appeared first on The Polyglot Developer .

## Browser statistics December 2017

*Henning Spjelkavik · Finn.no Tech*

Browser statistics, December 2017 at FINN.no#

Yuletide is coming real soon now, and we haven't published browser statistics for 2017 yet! The 2016 edition is here if you want to compare.

How many visitors use a desktop or laptop?

The trend continues, a higher percentage of the traffic is coming from a mobile terminal this year than in 2016, but the trend is that the growth of the curve seems to flatten.

Our first graph shows the share of our users using a mobile phone, tablet or a desktop computer to access FINN.no, regardless of whether they use our responsive web app (m.finn.no), or a native Android or iPhone app. 72% of our visits are now from a smartphone or a tablet. The traditional desktop/laptop has a market share of 28%.

Which offering/"app" of FINN are people using, regardless of whether the device is a desktop/laptop, mobile or tablet?

Given that a user is on a mobile or a tablet - are they using the web-version or the app? For the first time the majority of those accessing FINN from a mobile device use our native app - the www.finn.no share of the visits is down to 48%.

These graphs also shows that we changed the domain name again this spring, so that all web traffic is served from www.finn.no , and not m.finn.no any more.

Browsers and devices

The browser war is now only a memory of the veterans, but we still have to debug bugs in specific browsers from time to time, and it's useful to have an idea of which browsers could or should be ignored.

Let's also look at the which de-

**IN BRIEF**

**Axel Rauschmayer (2ality), Axel Rauschmayer (2ality):** In this chapter we develop a small web app in the same way that large professional web apps are developed:

We use libraries that we install via npm.

We write tests for some of the functionality.

We combine all JavaScript code into a single file before we serve the web app. That is called bundling . (Why we do that it explained later.)

**Nic Raboy, Nic Raboy (polyglot developer):** In a previous article I explained how to use Font Awesome glyph icons in your native Android application. Ionic Framework ships with IonicIcons included, but what if you wanted to include Font Awesome or any other font for that matter?

The following will show you how to include Font Awesome into your Android and iOS Ionic Framework project.

The post Use Font Awesome Glyph Icons With Ionic Framework appeared first on The Polyglot Developer .

**Axel Rauschmayer (2ality), Axel Rauschmayer (2ality):** In this blog post, we look at how WebAssembly has become an ecosystem for many programming languages and what technologies enable that.

**Nic Raboy, Nic Raboy (polyglot developer):** When you release your mobile Android or iOS application to the various app stores you'll be asked to submit a video for your app. Although optional, it is a great idea to do one in order to boost your app store optimization (ASO). So how do you get a video demo of your app? Using tools you probably already have installed you can create a screencast of your iOS or Android application.

We're going to see how to create a screencast for both platforms.

The post Create An iOS Or Android Device Screencast appeared first on The Polyglot Developer .

**Nic Raboy, Nic Raboy (polyglot developer):** Sometimes the best examples towards learning a new framework is through a simple user login sample. Login involves many concepts, including forms, data binding, routing, and potentially HTTP to a remote service, all of which are core concepts in any web application.

We're going to see how to implement a simple login form in a web application that uses the Vue.js JavaScript framework.

The post Simple User Login in a Vue.js Web Application appeared first on The Polyglot Developer .

## Manage Passwords With GPG, The Command Line, And Pass

*Nic Raboy · Nic Raboy (polyglot developer)*

There are a lot of password managers on the market, some in the cloud, some local, all with features that may or may not be useful in all circumstances. I'm personally an advocate of being in control of your secure information and shedding reliance on closed source or cloud alternatives. This is why I use pass , the standard unix password manager.

The pass application is Mac and Linux compatible, but Windows support probably isn't impossible. The application works by maintaining a list of password files that have been encrypted using GPG, a widely used cryptography software. Decrypting the files will result in access to your password information.

We're going to take a look at using pass and see why it is a convenient option for password management.

The post Manage Passwords With GPG, The Command Line, And Pass appeared first on The Polyglot Developer .

## Use Node.js And A Raspberry Pi Zero W To Scan For BLE iBeacon Devices

*Nic Raboy · Nic Raboy (polyglot developer)*

Earlier this month I had written a tutorial for detecting nearby BLE iBeacon devices using a Raspberry Pi Zero W and an application written with Golang. It was a great example of accomplishing something with Go and very little code.

Scanning for BLE devices is a great use case for Internet of Things (IoT) devices like the Raspberry Pi Zero W, and Golang isn't the only great language around. I, like many others, do a lot of Node.js development as well.

We're going to see how to scan for BLE iBeacon devices using Node.js and the popular Node.js BLE (Noble) library.

The post Use Node.js And A Raspberry Pi Zero W To Scan For BLE iBeacon Devices appeared first on The Polyglot Developer .

## Create And Sign Bitcoin Transactions With Golang

*Nic Raboy · Nic Raboy (polyglot developer)*

## Apache Cordova And Ionic Framework Apps Are Not Native Mobile Apps

*Nic Raboy · Nic Raboy (polyglot developer)*

If you've been keeping up with my content since the birth of The Polyglot Developer , you'll know that I was once a huge advocate of Apache Cordova development using frameworks like Ionic Framework. Having been a web developer and coming from native Android development with Java, cross-platform development using hybrid technologies seemed like a logical next step. Fast-forward to now, I'm no longer using Apache Cordova with Ionic Framework and have gone back to native development.

I recently came across an article by Ionic's CEO, Max Lynch, titled, Cordova/Ionic Apps are Native Apps , trying to explain that Ionic applications are native mobile applications. There are some valid points made in this article, but as someone who spent several years using the technology as well as using applications built with the technology, it is not something I agree with as a whole.

The post Apache Cordova And Ionic Framework Apps Are Not Native Mobile Apps appeared first on The Polyglot Developer .

## My Activity Report For 2019

*Nic Raboy · Nic Raboy (polyglot developer)*

It has been another great year for technology and The Polyglot Developer . Like I've done in a 2018 activity report , and the years before it, I wanted to take a moment to reflect on all that was accomplished for the 2019 calendar year.

If you're unfamiliar with this kind of post, it is more or less a numbers report for the various things that happened throughout the year. Such things include blog, podcast, and YouTube metrics, as well as information around events and speaking engagements.

Not only is this an opportunity for me to keep track of things, but you can use it as an opportunity to learn about how I've conducted business and apply it towards your own.

The post My Activity Report For 2019 appeared first on The Polyglot Developer .

## Implement Social Media Sharing With Ionic Framework

*Nic Raboy · Nic Raboy (polyglot developer)*

About a month ago I had written about creating and importing private keys as well as generating public addresses for Bitcoin and several other cryptocurrencies using the Go programming language. This previous tutorial had more of an emphasis on creating cryptocurrency wallets with Golang than anything.

The next step in making Bitcoin and other cryptocurrencies useful is to be able to transfer or send them to other people. Sending Bitcoin is part of a process known as creating and broadcasting a transaction.

While we won't be actually broadcasting a transaction in this tutorial, we're going to figure out how to create an unsigned transaction, then sign it, using the Go programming language and some popular Bitcoin packages.

The post Create And Sign Bitcoin Transactions With Golang appeared first on The Polyglot Developer .

## Create A Simple Todo List App Using Ionic 2 For Android And iOS

*Nic Raboy · Nic Raboy (polyglot developer)*

I've created a few tutorials around Ionic 2 while it was in its early alpha stage up until now. These tutorials explain how to use the bits and pieces that the framework or Angular offers, but I never demonstrated how to make a functional application. Seeing how to put the pieces together makes a huge difference when learning a new technology.

We're going to see how to build a simple todo list type Android and iOS application using Ionic 2, Angular , and TypeScript.

The post Create A Simple Todo List App Using Ionic 2 For Android And iOS appeared first on The Polyglot Developer .

So you've made a great app, but need some help marketing it. Adding social media features to your application is a great way help spread the word, without actually doing anything. Social media can bring good traffic for you, the developer, and provide useful features to your user at the same time.

Take for example, you made a mobile app that finds funny pictures on the internet. You might want to add a share button that shares a particular funny picture on your users social media account while including reference to your app so all their friends can use it too.

The following will show you how to make use of social media sharing in your Android and iOS mobile application using Ionic Framework .

The post Implement Social Media Sharing With Ionic Framework appeared first on The Polyglot Developer .

## Building Amazon Alexa Skills With Node.js, Revisited

*Nic Raboy · Nic Raboy (polyglot developer)*

A little more than two years ago, when the Amazon Echo first started picking up steam and when I was first exposed to virtual assistants, I had written a tutorial around creating a Skill for Amazon Alexa using Node.js and simple JavaScript. In this tutorial titled, Create an Amazon Alexa Skill Using Node.js and AWS Lambda , we saw how to create intent functions and sample utterances in preparation for deployment on AWS Lambda. I later wrote a tutorial titled, Test Amazon Alexa Skills Offline with Mocha and Chai for Node.js , which focused on building unit tests for these Skills and their intent functions. Fast forward to now and a few things have changed in the realm of Skill development.

In this tutorial we're going to see how to build a Skill for Alexa powered devices using Node.js and test it using popular frameworks and libraries such as Mocha and Chai .

The post Building Amazon Alexa Skills With Node.js, Revisited appeared first on The Polyglot Developer .

## Spreading love and good vibrations on GitHub

*espen · Finn.no Tech*

We have finally made an effort to put some of our repos out on GitHub under the finn-no organization . It is not exactly filled with stuff right now, but we aim to put a lot more of our libraries and tools out here.

Currently there are a few repositories out there:

eventHub

Awesome Board[Gone]

eventHub

This is just a tiny piece of code which enables you to build loosely coupled components or applications by utilizing the publish-subscribe pattern . The eventHub is just an object which publishes event to those to registered event listeners functions.

We have written one-page JS applications using only the hub besides jQuery for DOM manipulation. It scales pretty well and it is pretty much what you'd need to create an application. Once your application grows to be quite large you might want to add some other architectural components, but for small to medium sized projects it is brilliant. It is used for the dashboard part for the Awesome Board[Gone].

The Awesome Board[Gone] is the application which has featured in two articles about visualizing quality. We have shown it a few times to different audiences and due to request for making it available we decided it was a good thing to share it on GitHub. There are some pieces missing from the version we run internally, but have patience the rest might still make it to a repo near you.

## Make HTTP Requests In An Ionic Android And iOS App

*Nic Raboy · Nic Raboy (polyglot developer)*

Anyone looking to build a mobile application is going to find themselves needing to make HTTP requests to some remote web service at some time. It is just how the modern web and modern app development process is now. Previously I had demonstrated how to make HTTP requests in an Ionic Framework 1 application , but since Ionic 2 is all the rage right now, we're going to switch gears and see how it is done in the latest framework version.

The bulk of this tutorial will be demonstrating how to make these web service requests in Angular since it is fairly different from the first AngularJS version.

The post Make HTTP Requests In An Ionic Android And iOS App appeared first on The Polyglot Developer .

## TPDP Episode #22: NoSQL Databases And The Flexibility Of A Non-Relational Model

*Nic Raboy · Nic Raboy (polyglot developer)*

I am pleased to announce that the latest episode of The Polyglot Developer Podcast is now available for download. In this episode titled, NoSQL Databases and the Flexibility of a Non-Relational Model , I'm joined by Matt Groves where we talk about use-cases for NoSQL versus relational databases and how to use NoSQL in your own applications.

Matt Groves and I used to work together at Couchbase , which is a NoSQL database company, and is by no means the focus of this episode. The focus is NoSQL in general and all the great things that you can do with it.

The post TPDP Episode #22: NoSQL Databases And The Flexibility Of A Non-Relational Model appeared first on The Polyglot Developer .

## Using Network Sockets With The Go Programming Language

*Nic Raboy · Nic Raboy (polyglot developer)*

A few months back I wrote about using websockets in a Golang application for communication with an Angular client web application. While very useful and simplistic, in many cases websockets won't be the means for real-time communication between applications. It is often easier or better to use standard TCP network sockets as an alternative. For example, if you're developing an online video game, it will likely communicate to the server using TCP sockets rather than websockets.

We're going to see how to create a basic chat application using the Go programming language. This chat application will have a server for listening and routing client communications and a client for sending and receiving messages from the server.

The post Using Network Sockets With The Go Programming Language appeared first on The Polyglot Developer .

## Browser statistics October 2016

*Henning Spjelkavik · Finn.no Tech*

Last year, we presented the browser statistics of FINN.no as of June 2015 . It's time for an update!

How many visitors use a desktop or laptop?

Our first graph shows the share of our users using a mobile phone, tablet or a desktop computer to access FINN.no, regardless of whether they use our responsive web app (m.finn.no), or a native Android or iPhone app. 66% of our visits are now from a smartphone or a tablet. The traditional desktop/laptop has a market share of 34%.

If we exclude the native app users, which are obviously on either Android or iOS, what is the distribution between table, mobile and "other" (desktop) on our web site?

Which FINN.no application do people use?

We have two major offerings of FINN.no - the responsive web (served from the domains m.finn.no and www.finn.no), and native apps for mobile devices. 24% of our visits are from our native apps, and 76% from our responsive web.

Since we're in the middle of a migration, some of our internal details leaks out in this graph; some parts of the finn.no traffic is served from the domain www.finn.no and the majority from the domain m.finn.no. They are supposed to be merged "real soon now".

First of all, let's take a look at the numbers of the browser vendors . The ranking is clear, Apple is the biggest, ahead of Google, Microsoft, Samsung and Mozilla. Opera is no longer in the top 5.

In case you wondered which browser is the biggest on the "desktop" here's the trend. (Ignore the orange and blue triangle markers)

The complete browser statistics , across all applications and devices are as follows:

It still seems like a good idea to make sure your website works well with Safari! The old giant Microsoft now has only the 5th spot with IE 11 (6.8%) and the 12th with Edge 13 (1.7%).

Mobile details

We got a request to give some more details about the mobile devices that uses FINN.no. Apple still has a strong position in Norway!

The most popular browsers on mobile devices:

Then the question is - what is the "Android Browser"? This is the break down of devices that has delivered traffic from the "Android Browser".

Similarily for the Samsung Browser:.

And finally the trend on mobile browsers, by vendor:

Which version of Android or iOS?

Our apps need to work well on several versions of Android and iOS. How fast are our users upgrading? This shows the distribution of operating systems among

## I wish I knew my consumers - Maven Reverse Dependency

*roar · Finn.no Tech*

At FINN.no being a developer fixing bugs in a library is a breeze. Getting every user of your library to use the fix, however, is a different story. How to know who to notify? I mean, I know my library's dependencies, but who "out there" has dependency to the component where I just fixed a bug? I wish. Enter maven-dependency-graph.

The idea was born on the plane back home from a Copenhagen hosted conference. Graph database. Download neo4j and start dabbling at a maven plugin. Flying time Copenhagen - Oslo was too short, all of a sudden.

From there, the idea slept for a couple of years. Until the need arose somewhere among the developers. With 100+ different applications running with common core services and libraries, everybody suddenly needed to know who depended on their code which had recently been bugfixed. So the old idea was dusted off and once more saw the light of day. We needed to upgrade the server installation and the API to neo4j - which took some time to grasp; but after some playing around, it became obvious and easy.

The idea was to have every project report its dependencies to a graph database, building the tree of dependencies on each commit. This constitutes one half of the plugin. Over time, all projects will have reported their dependencies, and from there on part two of the plugin comes into use. It will examine the reverse dependencies to the current maven project, and report all incoming dependencies to it in the maven log. Hey, presto! We now know who out there uses us! And even which version they are using, thanks to two different keys into the built-in lucene index engine.

The plugin is published on github @ Finn Technology's account . Feel free! @gardleopard and @roarjoh

Usage examples

Dependencies to current maven project:

mvn
no.finntech:dependency-mapper-maven-plugin:read

[INFO] Scanning for projects...

[INFO]

[INFO] ————————————————————

[INFO] Building greenpages thrift-client 3.4.5-SNAPSHOT

[INFO] ————————————————————

[INFO]

[INFO] — dependency-mapper-maven-plugin:1.0-SNAPSHOT:read (default-cli) @ commons-thrift-client —

[INFO] Resolving reverse dependencies

`

(umpteen lines skipped...)

`

[INFO] ————————————————————

*Since we're in the middle of a migration, some of our internal details leaks out in this graph; some parts of the finn.*

Henning Spjelkavik

## TPDP Episode #8: Asynchronous and Event-Based Programming with RxJS

*Nic Raboy · Nic Raboy (polyglot developer)*

When it comes to modern JavaScript development, there are a few different ways to handle asynchronous events or data. You could use promises and callbacks, but as great as they are, present certain limitations. This is where RxJS comes into play with its reactive programming model. In this episode of The Polyglot Developer Podcast, guest speaker Ben Lesh and I discuss RxJS and where it fits in modern JavaScript development, whether it be server-side or front-end.

Ben Lesh is a senior software engineer at the very popular entertainment streaming company, Netflix . One of Ben's projects at Netflix includes the development and maintenance of RxJS since it is heavily used by the company. In the eighth episode, Asynchronous and Event-Based Programming with RxJS we discuss everything from what is RxJS, how it was inspired, who is using it, and why you should use it over a few of the alternative methods. If you've ever heard of RxJava or Rx. NET, these projects share some similarities to RxJS.

The post TPDP Episode #8: Asynchronous and Event-Based Programming with RxJS appeared first on The Polyglot Developer .

## Accept Text Input from Users in a Phaser Game

*Nic Raboy · Nic Raboy (polyglot developer)*

When you create a game, you may find yourself needing to accept keyboard input from the user. I'm not talking about using the keyboard to control your player, but instead to accept text from the keyboard to be used as a username or for the in-game chat.

If you've been keeping up with the Phaser content releasing on the blog, you might remember the tutorial titled, Creating a Multiplayer Drawing Game with Phaser and MongoDB . This tutorial demonstrated taking user input to be used as the game information, but it did so with a bit of HTML hacking.

There are better ways!

In this tutorial we're going to see how to take user keyboard input directly within a Phaser 3.x game using embedded HTML and JavaScript, rather than HTML layered on top of the game.

The post Accept Text Input from Users in a Phaser Game appeared first on The Polyglot Developer .

## Object Pooling Sprites in a Phaser Game for Performance Gains

*Nic Raboy · Nic Raboy (polyglot developer)*

Performance is everything when it comes to video games. Lag and stutter due to dropped frames can easily ruin your game. This means that you have to be considerate of how you manage your game resources to prevent unnecessary operations and stress on the client computer or mobile devices running the game.

If you've got a game with a lot of items, enemies, etc., creating and destroying these sprites as necessary is an expensive task. Instead, it makes sense to create an object pool with a predefined number of preloaded sprites that are used or hidden as necessary. This means that instead of creating a sprite when you need it, you pull the sprite from the pool. When you're done with the sprite, instead of destroying it, you add it back to the pool. While this seems like a silly task, it can have huge performance gains within your game.

In this tutorial, we're going to see how to create and use an object pool for our sprites in a Phaser 3.x game.

The post Object Pooling Sprites in a Phaser Game for Performance Gains appeared first on The Polyglot Developer .

**Global Chronicle**

Vol. 1, No. 047 · 2026-03-30

*Curated and composed automatically.*